SOE104 Oregon Trail — oregon-api Reference

The complete contract between your code and the engine. You never see the engine's source; this document is the interface. Read §1 first: knowing WHEN each of your methods is called is half of the design.

Companion documents: the coursework brief (what to build, how it is marked) and the Engine Rules Spec (every game number). Both are on the platform's sign-in page.

---

0. What you submit

One or more `.java` files (or a single `.zip` of them), 1–20 files:

- **no `package` declarations** — your classes live in the default package;
- every file starts with `import edu.soe104.oregon.api.*;` (plus `import java.util.List;` where you use lists) — see the sample party;
- one public class per file, UTF-8;
- **exactly one** implementation of `PartyFactory`;
- `createMembers()` returns **3 to 5** members — any other count fails the run.

Compile-time classpath contains only `oregon-api` — if a class is not in this document (or `java.*`), you cannot use it. Banned APIs (files, network, reflection, threads, `java.util.Random`, environment reads…) reject the submission outright: brief §4.

---

1. When the engine calls you

```
before the journey
  1. createMembers()          — build your roster (3–5 members)
  2. buySupplies()            — one shopping trip (null = classic basket)

every turn (1..120, until mile 2000):
  3. strategy.decide(state)   — pace, rations, rest          ← FIRST
  4.   the wagon travels and the party eats
  5. resolver.resolve(event)  — only if an event struck
  6. m.act(situation)         — once per LIVING member, roster order ← LAST
  7.   bookkeeping, survival check
```

Three timing facts worth designing around:

- **The strategy decides before anything happens** in the turn; members act **after** travel, eating and the event. `situation.getCurrentEvent()` is this turn's event, *already resolved* — reacting to it is legal (DEFEND exists for exactly this).

- The resolver runs **between** the two: whatever it learns, your members can still react to the same turn.
- `decide` may return `null` (engine default: NORMAL pace, FULL rations), `act` may return `null` (that member idles), `buySupplies` may return `null` (classic basket). All are legal — all are usually wasted turns.

The shared-context pattern (recommended)

The three surfaces see different views (§7, §8, §9). To share memory or give orders across them, wire ONE plain object of your own through the factory:

```
public class MyFactory implements PartyFactory {
    private final Plans plans = new Plans();     // your class: fields + methods

    public List<Member> createMembers() {
        return List.of(new MyHunter("Wren", plans), new MyDoctor("Anna", plans));
    }
    public PartyStrategy strategy() { return new MyStrategy(plans); }
    public Resolver resolver()    { return new MyResolver(plans); }
}
```

Now the strategy can set `plans.mode = Plans.Mode.CONSERVE;` and every member reads it the **same turn** (they act later). The resolver can record `plans.boughtFood = 50;` and the hunter knows the larder changed. This is composition — objects holding objects — and it is the intended answer to "how do my classes talk to each other?"

---

2. `Action` — what a member does this turn

Returned from `act(Situation)`.

```
Action.of(Action.Type.HUNT);        // preferred style
new Action(Action.Type.HUNT);       // identical
```

Actions are **binary**: a member chooses *what* to do, never how hard — every yield, cost and perk is a fixed rule the engine applies. Only `null` types throw.

TYPE	EFFECT (EXACT)
`HUNT`	costs **1 ammo**; yields **7 food** (Hunter: **10**). Surplus beyond the wagon's 500-food capacity spoils (the ammo is still spent) — don't hunt a full larder.
`HEAL`	self **+10** HP; if the actor is a `Doctor`: instead treats the **most wounded living member** for **+15** (themself if they are the sickest). No effect on a starving turn.
`REST`	self **+3** HP. No effect on a starving turn.
`SCOUT`	**+3 miles** this turn (Scout: **+6**). No health cost.
`MOVE`	**+5 miles** this turn, self **−1 HP** — for everyone, Drivers included.
`DEFEND`	if a threatening event struck this turn: **every living member +3 HP**. Triggers **once per turn** however many members defend — one defender is enough, five defenders waste four turns.

Members act in roster order; SCOUT/MOVE miles from all members add up.

---

3. `Situation` — the scout's morning briefing

Immutable; handed to every `act()` call. All getters:

METHOD	RETURNS
`getTurn()`	current turn, 1..120
`getDistanceSoFar()` / `getTotalDistance()`	miles travelled / 2000
`isThreatened()`	did an event strike this turn?
`getThreatLevel()`	its severity, 1–10 (0 if none)
`getPartyHealthAverage()`	mean health of living members, 0–100
`getFoodLevel()`	food in the wagon (units)
`getAmmoLevel()`	ammunition in the wagon
`getOxenCount()`	oxen pulling (0 = stranded, 4+ = full pace)
`getPartySize()`	living members this turn
`getCurrentEvent()` / `hasEvent()`	this turn's (already resolved) event

Per-turn consumption, so you can plan: `ration × partySize + 2 × oxen` food.

Worked example — the food runway (the number every strategy wants), using only Situation getters:

```
int perTurn = 2 * situation.getPartySize() + 2 * situation.getOxenCount();
int runway  = situation.getFoodLevel() / Math.max(1, perTurn);   // turns of food left
if (runway < 3) return Action.of(Action.Type.HUNT);               // all-hands hunting
```

The strategy-side version of the same calculation (from `PartyState`) is in §6.

---

4. `Member` and the five role classes

```
public abstract class Member implements Actable {
    public static final int MAX_HEALTH = 100;
    protected Member(String name);              // full health
    protected Member(String name, int hp);      // 0..100; outside range throws
    public abstract Action act(Situation situation);   // ← your behaviour
    public final String getName();
    public final int getHealth();
    public final boolean isAlive();
}
```

- You **must** override `act`; everything else is `final` — state is encapsulated by construction: health changes happen **only through the actions you return** (directly modifying engine-owned state is banned). Add your own **private** fields (thresholds, memory of past turns); public fields cost OOP marks.
- The five role classes are empty subclasses of `Member` with the same two constructors — extend one to receive its perk. Hunter, Scout and Doctor perks change what YOUR OWN actions do; Mechanic and Driver perks are **auras** the engine applies while the member lives (no code needed — roster them and keep them alive):

ROLE	PERK (EXACT)
`Hunter`	your HUNT yields 10 food per shot instead of 7
`Doctor`	your HEAL restores +15 base (not +10) **and** treats the most wounded member instead of only you
`Mechanic`	aura: while any Mechanic lives, BREAKDOWN **SAFE** repairs consume no spare part
`Scout`	your SCOUT finds +6 miles (not +3)
`Driver`	aura: while any Driver lives, GRUELING costs 2 HP per member (not 4) and the ox-death chance is 7% (not 15%)

- Extending `Member` directly is legal — every action works, no perk.
- Duplicate roles are fine: self-perk copies each boost their own actions (two Hunters each hunt at 10/ammo; they do not boost each other); auras do not stack — one living Mechanic or Driver is enough.

5. `PartyFactory` — your entry point

```
public interface PartyFactory {
    List<Member> createMembers();                    // 3–5 members (required)
    default List<Purchase> buySupplies() { … }       // null = classic basket
    default PartyStrategy strategy() { … }           // null = NORMAL/FULL default
    default Resolver resolver() { … }                // null = always SAFE
}
```

The General Store: every team gets a **flat $2,000 wallet**. Your purchases are applied in list order, each clipped to what the wallet still affords (food also clips to the 500-unit wagon capacity); **unspent wallet is your starting money**. `null` (or an empty list) buys the classic basket: 500 food, 80 ammo, 5 oxen, 2 parts — $300 change. Prices: food $2, ammo $3, parts $30, oxen $80.

```java
@Override
public List<Purchase> buySupplies() {
    return List.of(new Purchase(ResourceType.OXEN, 5),
                   new Purchase(ResourceType.FOOD, 500),
                   new Purchase(ResourceType.AMMO, 80),
                   new Purchase(ResourceType.SPARE_PARTS, 2));
}
```

`Purchase` = `new Purchase(good, quantity)`; money cannot be bought; negative quantity throws.

---

6. `PartyStrategy` and `PartyState`

```java
public interface PartyStrategy { Directive decide(PartyState state); }
```

`PartyState` — the whole-party view (richer than `Situation`):

METHOD	RETURNS
`getTurn()`, `getDistanceSoFar()`, `getTotalDistance()`	as in Situation
`getMembers()`	`List<Member>` — read names and health of everyone
`getResources()`	the wagon `Inventory` snapshot: `amountOf(type)`, `totalUnits()` — mutations you make to it do NOT affect the wagon (it is a copy for reading)
`getLastDirective()`	the directive actually used last turn
`aliveMembers()`	count of living members

A one-method interface — a lambda works: `return state -> new Directive(…);`

The runway worked example, strategy-side:

```java
@Override
public Directive decide(PartyState state) {
    int food  = state.getResources().amountOf(ResourceType.FOOD);
    int alive = state.aliveMembers();
    int oxen  = state.getResources().amountOf(ResourceType.OXEN);

    int perTurn = 2 * alive + 2 * oxen;      // at MEAGER rations (2/member)
    int runway  = perTurn == 0 ? 99 : food / perTurn;

    Directive.Ration ration = runway < 4 ? Directive.Ration.BARE
                            : runway > 12 ? Directive.Ration.FULL
                            : Directive.Ration.MEAGER;
    return new Directive(Directive.Pace.NORMAL, ration);
}
```

---

7. `Directive` — pace, rations, rest

```
new Directive(Directive.Pace.GRUELING, Directive.Ration.MEAGER);    // pace, rations
Directive.normal()                                                    // NORMAL + FULL + 0
```

DIAL	OPTIONS (EXACT)
Pace	`SLOW` 12 / `NORMAL` 20 / `GRUELING` 28 miles per turn, at 4+ oxen. Fewer oxen scale down proportionally; 0 oxen = no travel. GRUELING: every member −4 HP per turn, 15% chance an ox dies — **halved while a Driver lives** (2 HP, 7%).
Rations	`BARE` 1 (slow attrition: −1 HP per member per turn) / `MEAGER` 2 / `FULL` 5 food per living member per turn. FULL and fully fed → everyone +2 HP.

`null` or a thrown exception → the engine default for that turn (NORMAL + FULL). FULL is comfortable and expensive — the default strategy starves eventually.

---

8. `Resolver`, `Event`, `Resolution` — answering the trail

```
public interface Resolver { Resolution resolve(Event event); }
```

`Event` — everything you can read:

METHOD	RETURNS
`kind()`	one of `"RIVER"` `"STORM"` `"BREAKDOWN"` `"ILLNESS"` `"ATTACK"` `"TRADER"` `"HUNT_OPP"` `"BAD_TRAIL"` `"THEFT"`
`severity()`	1–10 (rises with progress; for markets it is the price level)
`getTurn()`, `getDescription()`	when / human-readable text
`isMarket()`	true on trading posts
`buyPrice(good)`, `sellPrice(good)`	posted prices (markets only)

The resolver **cannot see the wagon** (no inventory view) — purchases must be sized conservatively, or coordinated through the shared-context pattern (§1).

`Resolution` — everything you can answer:

```
Resolution.of(Resolution.Choice.SAFE);            // cautious: costs money or goods
Resolution.of(Resolution.Choice.RISKY);           // gamble: can pay big, can hurt, may
        kill an ox
Resolution.of(Resolution.Choice.AVOID);           // stay clear: usually costs miles
Resolution.buy(ResourceType.FOOD, 50);            // at the posted price
Resolution.sell(ResourceType.AMMO, 30);           // at 70% of the buy price
Resolution.haggleBuy(ResourceType.FOOD, 50);      // success chance 0.5 + (severity−5)*0.04
        [0.2..0.9]
Resolution.haggleSell(ResourceType.AMMO, 30);     // success: buy ×0.7 / sell ×1.3;
        failure: no deal
Resolution.decline();                             // browse and walk away
```

The event catalogue

`s` = severity (1–10). All HP damage hits every living member. What each posture does, per event: ### 🌊 RIVER — a river blocks the way (severity = depth)

- **SAFE** — pay the ferryman: **−$25** (must have it; if you cannot afford it, this choice is treated as AVOID)
- **RISKY** — ford the river: `s/10` chance the party takes **−10 HP** and **−20 food**; then a **20%** chance an ox drowns
- **AVOID** — wait on the bank: **−5 miles**

⛈️ STORM — bad weather rolls in

- **SAFE** — shelter and ride it out: party **−2 HP**
- **RISKY** — push through: party **−s HP**; **15%** chance an ox dies
- **AVOID** — go around: **−10 miles**

🛠️ BREAKDOWN — something on the wagon cracks

- **SAFE** — repair: consumes **1 spare part** — but with a living **Mechanic** the repair costs no part; with no parts left, same as AVOID (**−15 miles**)
- **RISKY** — jury-rig: `s/10` chance the party takes **−5 HP** and **−1 part**
- **AVOID** — abandon weight: **−15 miles**

🤒 ILLNESS — sickness sweeps the party

- **SAFE** — rest it off: **−3 × living members food**, **−10 miles**
- **RISKY** — push through: `s/10` chance the party takes **−(s+5) HP**, otherwise shakes it off entirely
- **AVOID** — endure it: party **−s HP**, sure thing

🐴 ATTACK — bandits or wildlife

- **SAFE** — flee: **−$30** (must have it; otherwise treated as AVOID) and **−10 food**
- **RISKY** — fight: party **−s HP**
- **AVOID** — hide: **−15 miles**

TRADER — a market opens

- **SAFE** — trade at posted prices: `buy(good, qty)` (clipped to money, **≤50 units/good/visit**, and food to wagon capacity)
- **RISKY** — haggle: success chance `0.5 + (s−5)×0.04` (clamped 0.2–0.9); success → buy ×0.7 / sell ×1.3; failure → no deal
- **AVOID** — walk away

🦌 HUNT_OPP — abundant game spotted

- **SAFE** — careful hunt: **−2 ammo** → **+20 food** (needs the ammo)
- **RISKY** — big hunt: **−4 ammo** → **+50 food**, `s/10` chance the party takes **−5 HP** (needs the ammo)
- **AVOID** — pass it up

🏔️ BAD_TRAIL — the trail forks

- **SAFE** — long detour: **−10 miles** and **−5 food** (the long road eats supplies)

- **RISKY** — shortcut gamble: `s/10` chance the party takes **−s HP** and **15%** an ox dies; otherwise **+5 miles**
- **AVOID** — turn back: **−20 miles**

🥷 THEFT — a thief circles the camp

- **SAFE** — post guards: **−10 food**
- **RISKY** — chase: 50/50 — party **−5 HP**, or **−20 food** and **−$20**
- **AVOID** — let them take the cash box: **−$15**

All event HP damage hits **every living member**. Food losses clip to stock; money price tags follow the affordability rule above.

Market rules: one transaction per visit; buys clip to what you can afford **and** to 50 units per good per visit (and food to wagon capacity); sells clip to stock. A SAFE option you cannot afford (ferry $25, fleeing bandits $30) is automatically downgraded to AVOID. The full event × posture × severity matrix is in the Engine Rules Spec — the game is fully transparent.

---

9. `ResourceType` and `Inventory`

```
public enum ResourceType { FOOD, OXEN, AMMO, SPARE_PARTS, MONEY }
```

`Inventory` (as handed to your strategy — a read-only copy): `amountOf(type)`, `totalUnits()` (sum of everything but is a rough wealth proxy). Members do not receive the inventory; they see food, ammo and oxen through `Situation` (§3).

---

10. Pitfalls checklist

- A `null` action type **throws** and caps your whole run at 40 — never build actions from unvalidated values.
- Two `PartyFactory` implementations → not graded. 1–2 or 6+ members → run fails. 0 or 21+ files → rejected.
- DEFEND does not stack — one defender per turn is enough; the rest should do something else.
- HEAL/REST do nothing on a starving turn (HUNT still works).
- The wagon carries at most 500 food — hunting a full larder wastes ammo.
- `getResources()` in your strategy is a snapshot: mutating it changes nothing (and pretending otherwise is not a strategy).
- No `java.util.Random`, no file/network/reflection/thread APIs — the brief §4 list is enforced by a bytecode scan before anything runs.

---

11. Numbers you will reuse

CONSTANT	VALUE
Journey	2000 miles, 120 turns
Wallet	$2,000; store prices food 2 / ammo 3 / parts 30 / oxen 80
Classic basket	500 food, 80 ammo, 5 oxen, 2 parts, $300 change
Wagon capacity	500 food
Market	buy cap 50/good/visit; sell = 70% of buy; bands ×0.7 / ×1.0 / ×1.4 by severity
Hunting	1 ammo → 7 food (Hunter 10)
Healing	+10 self (Doctor +15, most-wounded)
Rest	+3 self
Scout/Move	+3 / +5 miles, MOVE −1 HP for everyone; a Scout's SCOUT is +6
Defend	+3 to all living, once per threatened turn
Pace	12 / 20 / 28 miles (grueling: −4 HP each, 15% ox loss)
Rations	BARE 1 (−1 HP attrition) / MEAGER 2 / FULL 5 (+2 HP when fully fed)
Oxen	eat 2 food each per turn; 4+ = full pace; 0 = stranded
Starvation	5, 8, 12, 17, then 23 per turn to everyone; blocks healing
Events	15% early → 43% late; severity 1–10 rising